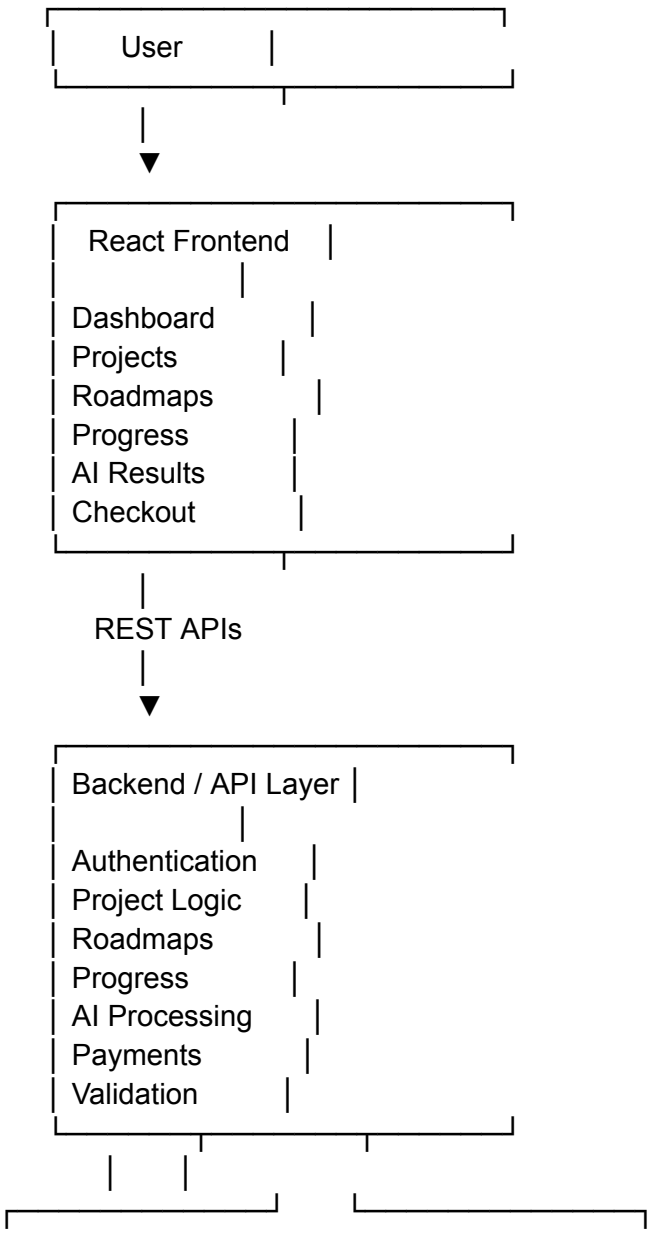
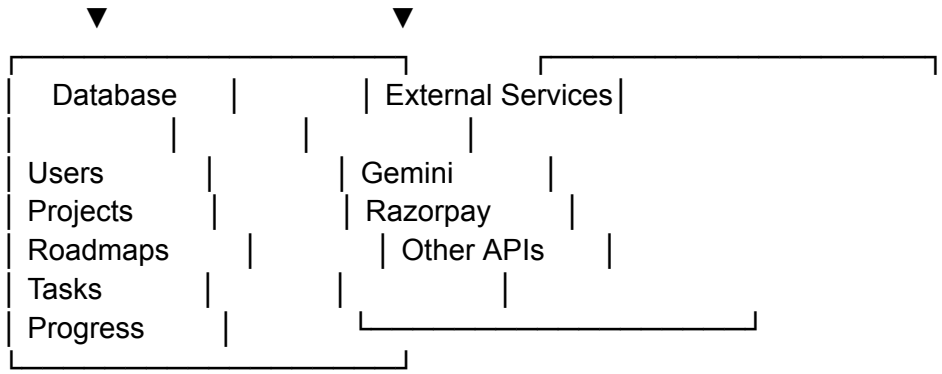
Bexra Technical Architecture

Overview

Bexra follows a client-server architecture where the frontend communicates with backend APIs, which handle application logic, data access, AI integrations, and external services.

At a high level:





Frontend Layer

The frontend provides the user-facing application.

Major areas include:

- Authentication
- User onboarding
- Dashboard
- Project management
- AI-generated business reports
- Roadmaps
- Tasks
- Progress tracking
- Help/support
- Upgrade and checkout

The frontend is responsible for presenting application state to the user and communicating with backend APIs.

Typical frontend flow:

User Action
↓
Frontend Component
↓
API Request
↓
Backend Response
↓
Application State Update
↓

UI Update

Backend Layer

The backend provides the application logic and API layer between the frontend and external services.

Responsibilities include:

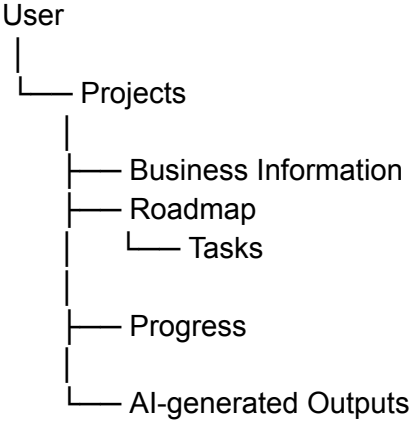
- Authentication
- Request validation
- Project-specific operations
- AI processing
- Database operations
- Progress management
- Payment workflows
- Error handling
- API security
- External service communication

The backend prevents important application logic from being handled entirely in the browser.

Database Layer

Bexra uses database-backed application state to store information associated with users and their projects.

Conceptually, the data model includes entities such as:



Project-scoped data allows different startup projects to maintain their own information and progress.

AI Integration

AI is integrated into the product through backend-controlled workflows.

A simplified AI request flow is:

User
↓
Frontend
↓
Backend API
↓
Project Information
↓
Gemini API
↓
AI-generated Analysis
↓
Backend Processing / Validation
↓
Frontend
↓
Structured Result

The backend acts as the controlled integration layer rather than exposing sensitive AI configuration directly to the client.

Business Analysis Workflow

A typical analysis workflow can be represented as:

Startup Project
↓
Project Information
↓
Analysis Request
↓

Backend
↓
AI Processing
↓
Structured Business Output
↓
Validation / Consistency Checks
↓
Report
↓
User Dashboard

Depending on the workflow, generated outputs can include:

- Executive Summary
- Market Analysis
- Competitor Analysis
- SWOT Analysis
- Financial Projections
- Industry Trends
- Recommended Next Steps

Project-Scoped Architecture

Bexra uses project-scoped functionality for features that depend on a particular startup.

Conceptually:

User
│
├── Project A
│ ├── Roadmap
│ ├── Tasks
│ ├── Progress
│ └── AI Reports
│
└── Project B
├── Roadmap
├── Tasks
├── Progress
└── AI Reports

This ensures that application requests operate on the intended project.

Project identifiers are validated before being used by relevant APIs.

Authentication Flow

A simplified authentication flow is:

User
↓
Login
↓
Authentication API
↓
Credential Validation
↓
Authenticated Session
↓
Protected Application Features

Authentication is used to ensure that application functionality is associated with the correct user.

Payment Flow

The payment workflow involves the application frontend, backend, and Razorpay.

Simplified flow:

User
↓
Upgrade
↓
Frontend Checkout
↓
Backend / Razorpay
↓
Payment
↓
Payment Verification
↓
Backend Confirmation
↓

Updated User Access

Payment completion should be verified on the backend rather than trusting only the client-side result.

Validation & Reliability

Bexra includes validation at multiple points in the application.

Examples include:

- Input validation
- Project ID validation
- API response validation
- Authentication validation
- Payment verification
- AI output consistency checks
- Error handling
- Idempotency-related handling

An important design principle is to avoid unnecessary AI requests when a deterministic validation or consistency check can solve the problem.

External Services

The product integrates with external services including:

Service	Purpose
Gemini	AI-powered analysis and guidance
Supabase	Database/authentication-related infrastructure
Razorpay	Payment and checkout functionality
Firebase	Supporting application infrastructure where applicable
Render	Backend/application deployment where applicable

Engineering Considerations

The architecture has been developed with several practical concerns in mind:

Security

Sensitive credentials and API keys should remain server-side and outside the public client application.

Reliability

External API failures need to be handled without causing the entire application flow to fail unexpectedly.

Validation

User input and project identifiers should be validated before processing.

Data Isolation

Project-specific operations should operate on the correct user's project.

Maintainability

Features are organized into separate frontend, backend, API, and data responsibilities rather than placing all application logic in a single layer.

Architecture Goal

The overall architecture is designed to support a SaaS product where users can create and manage startup projects, receive AI-powered analysis, follow structured roadmaps, track progress, and access paid functionality through a controlled application workflow.